3. AI large model voice interaction

3. AI large model voice interaction
- 1. Concept introduction
  - 1.1 What is "AI large model voice interaction"?
  - 1.2 Brief description of the implementation principle
- 2. Code analysis
  - Key code
    - 1. Voice input node (`largemodel/largemodel/asr.py`)
    - 2. AI service and voice output node (`largemodel/largemodel/model_service.py`)
  - Code analysis
- 3. Hands-on operation
  - 3.1 Configure the voice interaction
    - 3.1.1 Configure the platform and the voice backends (`yahboom.yaml`)
    - 3.1.2 Configure the model interface (`large_model_interface.yaml`)
  - 3.2 Start it and test the function

1. Concept introduction

1.1 What is "AI large model voice interaction"?

In the `largemodel` project, **AI large model voice interaction** connects the speech recognition (ASR), the large language model (LLM) and the speech synthesis (TTS) into a complete conversation system that can hear, can speak and can think. The ASR, the LLM and the TTS can each choose an online or an offline backend.

The user can hold a natural language conversation with the robot by voice; the robot understands the question, generates the answer and announces it through the speaker. With all the offline backends no access to the cloud is needed; with any of the online backends the network and the credentials of the corresponding service are needed.

The core of this function is the `model_service` **ROS2 node**. It plays the role of the brain and the nervous system: it subscribes to the recognition result of the ASR, calls the LLM to think, and then publishes the text reply of the LLM to the TTS node that synthesizes the speech.

1.2 Brief description of the implementation principle

The implementation principle of this function is a classic data flow pipeline:

1. **Voice -> text (ASR)**: the `asr` node keeps listening to the ambient sound; as soon as it detects that the user has finished a sentence, it converts it into text and publishes it on the `/asr_text` topic.
2. **Text -> thinking -> text (LLM)**: the `model_service` node subscribes to the `/asr_text` topic. When it receives the text from the ASR, it passes it as a prompt to the locally deployed large language model (such as `Qwen` running through `Ollama`). The LLM generates the text of the answer according to the context.
3. **Text -> voice (TTS)**: after the `model_service` node gets the answer of the LLM, it publishes it on the `/tts_text` topic.
4. **Voice playback**: the TTS node subscribes to the text to announce, calls the offline or the online speech

synthesis service according to the configuration, and plays it through the speaker.

This flow forms a complete **voice input -> text processing -> text output -> voice output** closed loop.

---

2. Code analysis

Key code

1. Voice input node (`largemodel/largemodel/asr.py`)

```
# From largemodel/largemodel/asr.py
class ASRNode(Node):
    def __init__(self):
        # ...
        self.asr_pub = self.create_publisher(String, "asr", 5)
        # ...

    def kws_handler(self)->None:
        if self.listen_for_speech(self.mic_index):
            asr_text = self.ASR_conversion(self.user_speechdir)
            if asr_text != 'error':
                self.asr_pub_result(asr_text)

    def asr_pub_result(self,asr_result:str)->None:
        msg=String(data=asr_result)
        self.asr_pub.publish(msg)
```

2. AI service and voice output node (`largemodel/largemodel/model_service.py`)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.asrsub = self.create_subscription(String,'asr', self.asr_callback,1)
        # ...

    def asr_callback(self,msg):
        """Callback function for handling the ASR messages."""
        # ...
        result = self.model_client.infer_with_text(msg.data, message=messages_to_use)
        self.process_model_result(result)

    def process_model_result(self, result, from_seewhat=False):
        """Process the result returned by the model."""
        # ...
        user_friendly_response = response_json.get("response", "Processing...")
        # ...
        self._safe_play_audio(user_friendly_response)
        # ...
        self.execute_tools(tools_list)
```

```
    def _safe_play_audio(self, text_to_speak: str):
        """
        Synthesizes and plays all non-empty messages only in non-text chat mode.
        """
        if not self.text_chat_mode and text_to_speak:
            try:
                self.model_client.voice_synthesis(text_to_speak, self.tts_out_path)
                self.play_audio_async(self.tts_out_path)
            except Exception as e:
                self.get_logger().error(f"Safe audio playback failed: {e}")
```

Code analysis

The voice interaction function of the AI large model is completed collaboratively by two independent ROS nodes, `asr.py` and `model_service.py`. They communicate with each other through the ROS topic `/asr`, which forms a complete processing loop.

1. **Voice input and publishing (`asr.py`)**:
    - The function of the `ASRNode` node is to handle the voice input. In the `kws_handler` function it completes the recording and the text conversion.
    - After the conversion succeeds, the `asr_pub_result` function is called. This function puts the resulting text string into a `std_msgs.msg.String` message and then publishes that message to the `/asr` topic through the `self.asr_pub` publisher.
2. **Subscription and response (`model_service.py`)**:
    - When it is initialized, the `LargeModelService` node subscribes to the `/asr` topic through the `create_subscription` method and specifies `asr_callback` as its callback function.
    - When the `ASRNode` publishes a message on the `/asr` topic, the ROS system automatically calls the `asr_callback` function of the `LargeModelService` and passes the message in as the parameter `msg`.
3. **The interaction loop**:
    - After the `asr_callback` function receives the message, it extracts the text data in it (`msg.data`) and passes it to the large model for the inference (`self.model_client.infer_with_text`).
    - When the inference is done, the `process_model_result` function is called. This function extracts the text to be used for the reply from the model result.
    - Then the `_safe_play_audio` function is called; it uses the TTS function (`self.model_client.voice_synthesis`) to synthesize the reply text into audio and play it.
    - At this point a complete interaction loop from "receiving the user voice" to "answering the user with the voice" has been completed. The two nodes are decoupled through the publish-subscribe pattern of ROS: `asr.py` is responsible for the input, and `model_service.py` for the processing and the output.

3. Hands-on operation

3.1 Configure the voice interaction

The voice interaction consists of three parts: the ASR, the LLM and the TTS. The ASR and the TTS can each choose online or offline; the LLM is selected separately with `llm_platform`. Modify the corresponding keys in the existing configuration file first, and do not add duplicates in the same parameter block.

3.1.1 Configure the platform and the voice backends (`yahboom.yaml`)

Open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Choose as needed in the existing parameter blocks:

```
asr:
  ros__parameters:
    use_oline_asr: False  # False: offline ASR; True: online ASR
    regional_setting: "International"

model_service:
  ros__parameters:
    language: 'en'
    useolinetts: False    # False: offline TTS; True: online TTS
    llm_platform: 'ollama' # 'ollama': offline LLM; 'tongyi': the online Tongyi
    regional_setting: "International"
```

For example, use `False`, `False`, `ollama` for everything offline, and `True`, `True`, `tongyi` for everything online. A mixed choice is also possible, for example an online LLM with an offline ASR/TTS. The online ASR or TTS also needs the credentials configured as described in articles 1 and 2 of this chapter; switching the boolean value alone does not generate or fill in the service provider keys.

3.1.2 Configure the model interface (`large_model_interface.yaml`)

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Online Tongyi LLM: fill in a valid API Key and an available model; if the ASR/TTS are online as well, add the service provider credentials and the model parameters of each as described in articles 1 and 2.

```
qianwen_api_key: "Replace it with your own Tongyi API-Key"
qianwen_model: "qwen-vl-max-latest"
```

Offline Ollama LLM: confirm that the service address is reachable and that the model has been downloaded; for the offline ASR/TTS confirm that the path of each model file exists.

```
ollama_host: "http://localhost:11434"
ollama_model: "qwen2.5:7b"  # Example; it has to match the model name in ollama list
local_asr_model: "/home/jetson/yahboom_ws/src/largemodel/MODELS/asr/SenseVoiceSmall"
en_tts_model: "/home/jetson/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-libritts-
high.onnx"
en_tts_json: "/home/jetson/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-libritts-
high.onnx.json"
```

Restart the voice program after saving the configuration. The online backends require the Jetson to be able to reach the service provider and the account to be usable; the offline backends require Ollama and the local voice models to be available. Do not publish a real API Key. The speed of the offline inference depends on the computing power of the Jetson mainboard and is usually slower than the online models;

3.2 Start it and test the function

1. **Start the `largemodel` main program**:
   Run the following command to start the voice interaction:

   ```
   ros2 launch largemodel largemodel_control.launch.py
   ```

2. **Test**:
   - **Wake up**: say "Hi, yahboom" to the microphone.
   - **Conversation**: after the speaker replies, you can say the question you want to ask.
   - **Watch the log**: in the terminal that runs the `launch` file you should see:
     1. The ASR node recognizes your question and prints it.
     2. The `model_service` node receives the text, calls the LLM and prints the reply of the LLM.
   - **Listen to the answer**: shortly afterwards you should hear the robot answer your question with the synthesized voice.